

Fundamentals of Data Structures

Project 3: Normal Dijkstra Sequence



Author:

Date: 2026/05/07

2025-2026 Spring & Summer Semester

Table of Contents

Chapter 1: Introduction	3
Chapter 2: Algorithm Specification	3
2.1. Data Structures	3
2.2. Main Program Sketch	4
2.3. Graph Construction with Parallel Edge Handling	4
2.4. Sequence Verification (Core Algorithm)	4
2.5. Summary of Complexities	5
Chapter 3: Testing Results	6
3.1. Sample Output Verification	6
3.2. Test Case Table	6
Chapter 4: Analysis and Comments	9
4.1. Time Complexity Analysis	9
4.2. Space Complexity Analysis	10
4.3. Design Decisions	10
4.4. Possible Improvements	10
Appendix: Source Code (in C)	11
Declaration	14

Chapter 1: Introduction

Dijkstra’s algorithm, conceived by computer scientist Edsger W. Dijkstra in 1956 and published three years later, is one of the most celebrated greedy algorithms in computer science. It solves the single-source shortest path problem: given a weighted graph with non-negative edge weights and a designated source vertex, the algorithm computes the shortest path from the source to every other vertex.

The algorithm works by maintaining a set of “collected” vertices whose shortest distances from the source have been finalized. At each step, it selects the uncollected vertex with the minimum current distance, adds it to the collected set, and uses its outgoing edges to potentially improve (relax) the tentative distances of its neighbors. The order in which vertices are collected forms what we call a **Dijkstra sequence**.

A key observation is that at any given step, there may be **multiple** uncollected vertices sharing the same minimum distance. Since Dijkstra’s algorithm does not specify a tie-breaking rule, any of these tied vertices is a valid choice, meaning a single graph may admit more than one valid Dijkstra sequence.

Problem Statement. Given a connected undirected weighted graph with N_v vertices ($N_v \leq 10^3$) and N_e edges ($N_e \leq 10^5$), and K query sequences ($K \leq 100$), each being a permutation of all N_v vertices, determine for each sequence whether it could have been produced by some execution of Dijkstra’s algorithm starting from the sequence’s first vertex. Output **Yes** if it is a valid Dijkstra sequence, or **No** otherwise.

Input format. Vertices are numbered 1 to N_v . The first line gives N_v and N_e ; the next N_e lines each describe an undirected edge (u, v, w) with positive weight $w \leq 100$. Then K query sequences follow, each a permutation of $1, \dots, N_v$.

Chapter 2: Algorithm Specification

2.1. Data Structures

The graph is stored as an **adjacency matrix** `graph[MAXN][MAXN]`, where `graph[u][v]` holds the weight of the edge between u and v , or ∞ (`0x3f3f3f3f`) if no edge exists. Given $N_v \leq 10^3$, the matrix occupies at most 10^6 integers (≈ 4 MB), which fits comfortably in memory. Using an adjacency matrix also gives $O(1)$ edge-weight lookup, which simplifies the relaxation step.

Three auxiliary arrays are used per query:

- `dist[v]`: the current shortest known distance from the source to vertex v .
- `collected[v]`: a boolean flag indicating whether vertex v has been finalized.
- `seq[v]`: the candidate sequence to be verified.

All arrays are declared globally to avoid stack overflow and are reinitialized before each query.

2.2. Main Program Sketch

The main program reads the graph once, then processes each query independently:

```

1 Read  $N_v, N_e$ 
2 InitGraph() (set all entries to  $\infty$ )
3 for  $i = 1$  to  $N_e$ :
4   | Read edge  $(u, v, w)$ ; store minimum weight in graph[u][v] and graph[v][u]
5 Read  $K$ 
6 for each of the  $K$  queries:
7   | Read sequence seq[0..Nv-1]
8   | if CheckSequence() then print Yes else print No

```

2.3. Graph Construction with Parallel Edge Handling

Since the input may contain multiple edges between the same pair of vertices, we initialize all entries to ∞ and keep only the minimum weight for each vertex pair:

```

1 function InitGraph():
2   | for  $i = 1$  to  $N_v$ :
3     | for  $j = 1$  to  $N_v$ :
4       | graph[i][j]  $\leftarrow$   $\infty$ 

1 for each edge  $(u, v, w)$ :
2   | if  $w < \text{graph}[u][v]$  then
3     | graph[u][v]  $\leftarrow$   $w$ 
4     | graph[v][u]  $\leftarrow$   $w$  (undirected graph)

```

This ensures that `graph[u][v]` always holds the shortest direct connection between u and v , which is essential for correct shortest-path computation.

Time complexity: $O(N_v^2)$ for initialization, $O(N_e)$ for reading edges.

2.4. Sequence Verification (Core Algorithm)

The central idea is to **simulate** Dijkstra's algorithm, but instead of choosing the minimum-distance vertex ourselves, we check whether the sequence's prescribed vertex is a valid choice at each step.

Initialization:

```
1 function CheckSequence():
2   for  $v = 1$  to  $N_v$ :
3      $\text{dist}[v] \leftarrow \infty$ ;  $\text{collected}[v] \leftarrow \text{false}$ 
4    $\text{source} \leftarrow \text{seq}[0]$ 
5    $\text{dist}[\text{source}] \leftarrow 0$ 
```

Main loop — for each step i from 0 to $N_v - 1$:

```
1  $v \leftarrow \text{seq}[i]$    (vertex prescribed by the candidate sequence)
2 Step A: Find global minimum
3    $\text{minDist} \leftarrow \infty$ 
4   for  $j = 1$  to  $N_v$ :
5     if not  $\text{collected}[j]$  and  $\text{dist}[j] < \text{minDist}$  then
6        $\text{minDist} \leftarrow \text{dist}[j]$ 
7 Step B: Validity check
8   if  $\text{dist}[v] \neq \text{minDist}$  then
9     return No   ( $v$  is not an optimal choice at this step)
10 Step C: Collect and relax
11    $\text{collected}[v] \leftarrow \text{true}$ 
12   for  $j = 1$  to  $N_v$ :
13     if not  $\text{collected}[j]$  and  $\text{graph}[v][j] \neq \infty$  then
14       if  $\text{dist}[v] + \text{graph}[v][j] < \text{dist}[j]$  then
15          $\text{dist}[j] \leftarrow \text{dist}[v] + \text{graph}[v][j]$ 
16 return Yes   (all  $N_v$  steps passed)
```

Correctness argument. At step i , Dijkstra's algorithm may legitimately collect any uncollected vertex whose **dist** value equals the current global minimum among all uncollected vertices. This is because Dijkstra's greedy choice only requires picking *some* vertex with the minimum distance — the algorithm is correct regardless of how ties are broken. Therefore, the sequence is valid if and only if, at every step i , **dist[seq[i]]** equals the minimum **dist** among all uncollected vertices. This is exactly what Step B verifies.

2.5. Summary of Complexities

Operation	Time	Space
Graph initialization	$O(N_v^2)$	$O(N_v^2)$ (adjacency matrix)
Read edges	$O(N_e)$	—

Operation	Time	Space
Verify one sequence	$O(N_v^2)$	$O(N_v)$ (dist + collected)
All K queries	$O(K \cdot N_v^2)$	$O(N_v)$ (reused per query)
Overall	$O(N_v^2 + N_e + K \cdot N_v^2)$	$O(N_v^2)$

Chapter 3: Testing Results

3.1. Sample Output Verification

The sample input and output are shown below for reference.

Sample Input		Sample Output	
1	5 7	1	Yes
2	1 2 2	2	Yes
3	1 5 1	3	Yes
4	2 3 1	4	No
5	2 4 1		
6	2 5 2		
7	3 5 1		
8	3 4 1		
9	4		
10	5 1 3 4 2		
11	5 3 1 2 4		
12	2 3 4 5 1		
13	3 2 1 5 4		

Trace of the fourth query (3 2 1 5 4 → No): Source is vertex 3. After collecting vertex 3 (dist = 0), relaxation gives dist[2] = 1, dist[5] = 1. The next vertex in the sequence is 2, which is valid (dist[2] = 1 = min). After collecting vertex 2, relaxation gives dist[1] = 3, dist[4] = 2. Now dist[5] = 1 remains the global minimum, but the sequence prescribes vertex 1 (dist[1] = 3 ≠ 1). Therefore the sequence is invalid.

3.2. Test Case Table

#	Description	Expected	Actual	Bug Cause (if any)	Status
1	Sample input: 5 vertices, 7 edges, 4 query sequences. Three valid orderings and one in-	Yes Yes Yes No	Yes Yes Yes No	N/A	Pass

#	Description	Expected	Actual	Bug Cause (if any)	Status
	valid (vertex 1 chosen when $\text{dist}[5]$ is still smaller). Exercises basic correctness and tie-breaking.				
2	Single-vertex graph ($N_v = 1$, $N_e = 0$). The only possible sequence is the source itself. Edge case: relaxation loop body never executes.	Yes	Yes	N/A	Pass
3	Linear chain 1–2–3 with unit weights. Sequence $1 \rightarrow 2 \rightarrow 3$ follows shortest-path order ($\text{dist}[2] = 1 < \text{dist}[3] = 2$). Sequence $1 \rightarrow 3 \rightarrow 2$ is invalid since $\text{dist}[2] = 1 < \text{dist}[3] = 2$ after collecting vertex 1.	Yes No	Yes No	N/A	Pass
4	Star-like graph with disparate weights: edge 1–2 has weight 1, edge 1–3 has weight 10. Sequence $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$ picks vertex 3 ($\text{dist} = 10$) before vertex 2 ($\text{dist} = 1$), violating the greedy choice.	No	No	N/A	Pass

#	Description	Expected	Actual	Bug Cause (if any)	Status
5	Tie-breaking: two edges from source with identical weight 5. Both $1 \rightarrow 2 \rightarrow 3$ and $1 \rightarrow 3 \rightarrow 2$ are valid since $\text{dist}[2] = \text{dist}[3] = 5$ after collecting the source, so either may be chosen next.	Yes Yes	Yes Yes	N/A	Pass
6	Parallel edges: two edges between vertices 1 and 2 with weights 10 and 3. Tests that the program correctly retains only the minimum weight (3).	Yes	Yes	N/A	Pass
7	Complete graph K_4 with all edge weights equal to 1. After collecting the source, all remaining vertices have $\text{dist} = 1$. Any permutation starting from vertex 1 should be valid (full three-way tie).	Yes Yes Yes	Yes Yes Yes	N/A	Pass
8	Chain 1-2-3-4 with weights 1, 100, 1. After collecting 1 and 2, $\text{dist}[3] = 101$ and $\text{dist}[4] = 102$. Sequence $1 \rightarrow 2 \rightarrow 4 \rightarrow 3$	Yes No	Yes No	N/A	Pass

#	Description	Expected	Actual	Bug Cause (if any)	Status
	is invalid because $\text{dist}[4] = 102 > \text{dist}[3] = 101$.				
9	Triangle with all edges weight 100. After collecting vertex 1, $\text{dist}[2] = \text{dist}[3] = 100$. Both orderings valid (ties). Tests large-weight tie-breaking.	Yes Yes	Yes Yes	N/A	Pass
10	Maximum scale stress test: Dense graph with $N_v = 1000$ and $N_e \approx 10^5$, and $K = 100$ query sequences. Tests if the borderline 10^8 operations complete within the 1-second time limit without timeout.	Accepted	Pass within 0.47s	N/A	Pass

All 10 test cases pass, covering the sample input, boundary cases (single vertex, parallel edges), tie-breaking scenarios (equal weights), invalid sequences with clear distance violations, and maximum-scale performance stress testing.

Chapter 4: Analysis and Comments

4.1. Time Complexity Analysis

The overall time complexity is $O(N_v^2 + N_e + K \cdot N_v^2)$, which simplifies to $O(K \cdot N_v^2)$ since $N_e \leq N_v^2$.

- **Graph construction** takes $O(N_v^2)$ for matrix initialization plus $O(N_e)$ for reading edges.
- **Each query verification** takes $O(N_v^2)$: the outer loop runs N_v iterations, and each iteration performs an $O(N_v)$ scan for the minimum distance plus an $O(N_v)$ relaxation pass over all potential neighbors.

- **All K queries** together take $O(K \cdot N_v^2)$, since each query reinitializes and runs independently.

With $N_v \leq 10^3$ and $K \leq 100$, the worst case is approximately 10^8 simple comparison and addition operations. This is borderline for a 1-second time limit, but acceptable because the inner loop consists of only lightweight integer operations with no memory allocation or function-call overhead.

4.2. Space Complexity Analysis

The space usage is dominated by the adjacency matrix at $O(N_v^2)$.

- Adjacency matrix `graph[MAXN][MAXN]`: 10^6 integers ≈ 4 MB.
- Auxiliary arrays `dist[]`, `collected[]`, `seq[]`: each $O(N_v)$.
- Total: $O(N_v^2)$.

All arrays are declared globally and reused across queries, so no dynamic memory allocation is performed.

4.3. Design Decisions

1. Adjacency matrix over adjacency list. For $N_v \leq 10^3$, the $O(N_v^2)$ space cost is manageable (≈ 4 MB). The adjacency matrix provides $O(1)$ edge-weight lookup, which simplifies the relaxation step (no need for linked-list traversal). It also naturally handles parallel edges by storing only the minimum weight.

2. Global arrays. Declaring `graph`, `dist`, `collected`, and `seq` as global variables avoids two issues: (a) stack overflow — local arrays of size 10^6 would exceed the typical $\approx 10^4$ KB stack limit; and (b) repeated allocation — global arrays persist across query iterations and need only be reinitialized, not reallocated.

3. Simulation-based verification. An alternative approach would be to run a full Dijkstra and compare final distances. However, the simulation approach is more direct: it verifies the sequence *as it unfolds*, which allows early termination on the first invalid step and avoids generating all possible valid sequences.

4. `0x3f3f3f3f` as infinity. This value ($\approx 10^9$) is safely larger than any possible shortest distance ($N_v \times w_{\max} = 10^3 \times 100 = 10^5$) while remaining far below `INT_MAX`, avoiding overflow when computing `dist[v] + graph[v][j]`.

4.4. Possible Improvements

Priority queue (min-heap): Replacing the $O(N_v)$ linear scan for the minimum distance with a binary min-heap would reduce each query's time from $O(N_v^2)$ to $O((N_v + N_e) \log N_v)$. For dense graphs this offers no advantage, but for sparse graphs ($N_e =$

$O(N_v)$), it would be significantly faster. A Fibonacci heap would further improve the amortized cost of decrease-key operations from $O(\log N_v)$ to $O(1)$.

Adjacency list for sparse graphs: If $N_e < N_v^2$, an adjacency list representation would reduce space from $O(N_v^2)$ to $O(N_v + N_e)$ and speed up the relaxation step from $O(N_v)$ to $O(\deg(v))$ per collected vertex.

Precomputation of shortest distances: If multiple sequences share the same source vertex, the shortest distances could be computed once and reused for all queries with that source. However, since each sequence may have a different first vertex (and hence a different source), this optimization has limited applicability in the current problem setting.

Appendix: Source Code (in C)

code/sol.c 

```

1  #include <stdio.h>
2
3  #define MAXN 1001      /* Maximum number of vertices (1-indexed) */
4  #define INF  0x3f3f3f3f /* Represents infinity (no direct edge) */
5
6  int Nv, Ne;            /* Number of vertices and edges */
7  int graph[MAXN][MAXN]; /* Adjacency matrix: graph[u][v] = edge weight, INF
   if no edge */
8  int dist[MAXN];        /* Shortest distance from source to each vertex */
9  int collected[MAXN];   /* collected[v] = 1 if vertex v has been finalized */
10 int seq[MAXN];          /* The candidate sequence to be verified */
11
12 /*
13  * initGraph: Initialize the adjacency matrix.
14  * Set all entries to INF to indicate no edges exist initially.
15  * Vertices are numbered from 1 to Nv.
16  */
17 void initGraph()
18 {
19     int i, j;
20     for (i = 1; i <= Nv; i++)
21         for (j = 1; j <= Nv; j++)
22             graph[i][j] = INF;
23 }
24
25 /*
26  * checkSequence: Verify whether seq[] is a valid Dijkstra sequence.
27  *
28  * Strategy: Simulate Dijkstra's algorithm.

```

```

29  * At each step i, instead of choosing the minimum ourselves, we check whether
30  * seq[i] (the vertex given by the candidate sequence) is a valid choice –
31  * i.e., its dist value equals the current global minimum among all uncollected
32  * vertices.
33  * If yes, collect it and relax neighbors. If no, the sequence is invalid.
34  *
35  * Returns 1 if valid ("Yes"), 0 if invalid ("No").
36  */
37  int checkSequence()
38  {
39      int i, j, v, minDist;
40
41      /* Initialize: all distances are INF, no vertex collected */
42      for (i = 1; i <= Nv; i++) {
43          dist[i] = INF;
44          collected[i] = 0;
45      }
46
47      /* The source vertex is the first element of the sequence */
48      int source = seq[0];
49      dist[source] = 0; /* Distance from source to itself is 0 */
50
51      /* Process each vertex in the given sequence order */
52      for (i = 0; i < Nv; i++) {
53          v = seq[i]; /* Candidate vertex to collect at step i */
54
55          /* Find the minimum dist among all uncollected vertices */
56          minDist = INF;
57          for (j = 1; j <= Nv; j++) {
58              if (!collected[j] && dist[j] < minDist)
59                  minDist = dist[j];
60          }
61
62          /* Key check: dist[v] must equal minDist.
63           * If dist[v] > minDist, then v is not an optimal choice at this step,
64           * meaning this sequence cannot be produced by any run of Dijkstra's
65           * algorithm.
66           */
67          if (dist[v] != minDist)
68              return 0; /* Invalid Dijkstra sequence */
69
70          /* Collect vertex v: mark it as finalized */
71          collected[v] = 1;

```

```

72      /*
73      * Relaxation: update distances of v's uncollected neighbors.
74      * For each neighbor j of v, if going through v gives a shorter path,
75      * update dist[j].
76      */
77      for (j = 1; j <= Nv; j++) {
78          if (!collected[j] && graph[v][j] != INF) {
79              if (dist[v] + graph[v][j] < dist[j])
80                  dist[j] = dist[v] + graph[v][j];
81          }
82      }
83
84      return 1; /* All steps passed: valid Dijkstra sequence */
85  }
86
87  int main()
88  {
89      int i, u, v, w, K;
90
91      /* Read the number of vertices and edges */
92      scanf("%d %d", &Nv, &Ne);
93      initGraph();
94
95      /*
96      * Read Ne edges. The graph is undirected, so each edge (u, v, w)
97      * is stored in both graph[u][v] and graph[v][u].
98      * If multiple edges exist between the same pair of vertices,
99      * keep only the minimum weight (greedy shortest-path preprocessing).
100     */
101     for (i = 0; i < Ne; i++) {
102         scanf("%d %d %d", &u, &v, &w);
103         if (w < graph[u][v]) {
104             graph[u][v] = w;
105             graph[v][u] = w;
106         }
107     }
108
109     /* Read and verify each query sequence */
110     scanf("%d", &K);
111     while (K--) {
112         /* Read a permutation of all Nv vertices */
113         for (i = 0; i < Nv; i++)
114             scanf("%d", &seq[i]);
115

```

```
116      /* Print "Yes" if it is a valid Dijkstra sequence, "No" otherwise */
117      if (checkSequence())
118          printf("Yes\n");
119      else
120          printf("No\n");
121  }
122
123      return 0;
124  }
```

Declaration

I hereby declare that all the work done in this project titled “Normal Dijkstra Sequence” is of my independent effort.